[1교시] 왜 Streamlit인가? & 핵심 동작 원리 (50분)

> **학습 목표**: Streamlit의 존재 이유를 이해하고, Data Flow(재실행) 원리를 체험하며, st.write()와 Magic Commands를 활용하여 다양한 데이터를 화면에 출력할 수 있다.

---

0. Streamlit이란? (5분)

정의

Streamlit은 Python만으로 데이터 웹 애플리케이션을 빠르게 만들 수 있는 **오픈소스 프레임워크**이다.

- 2019년 출시, 2022년 Snowflake에 인수(합류)
- GitHub Stars 수만 개 이상 (활발한 오픈소스 커뮤니티)
- 슬로건: **"데이터 앱을 만들고 공유하는 가장 빠른 방법"**

누가, 왜 만들었나?

Streamlit의 창시자 아드리엔 트뢰유(Adrien Treuille)는 전직 Google/CMU 연구원으로, 데이터 과학자들이 겪는 공통 문제를 해결하기 위해 만들었다:

> "분석은 끝났는데, 결과를 보여줄 방법이 없다"

기존 방식	문제점
Jupyter Notebook 공유	비개발자가 실행하기 어려움
엑셀/PPT로 캡처	상호작용 불가, 업데이트 번거로움
Flask/Django로 웹 개발	HTML/CSS/JS 필요, 시간 소요 큼

Streamlit은 이 모든 문제를 **"Python 스크립트 하나"**로 해결한다.

핵심 특징 요약

특징	설명
Pure Python	HTML, CSS, JS 없이 Python만으로 UI 구성
실시간 반영	코드를 저장하면 브라우저에 즉시 반영
위젯 내장	슬라이더, 버튼, 셀렉트박스 등을 한 줄로 생성
차트 내장	`st.line_chart()`, `st.bar_chart()` 등 시각화 즉시 가능
무료 배포	Streamlit Community Cloud로 무료 웹 배포 지원

어디에 쓰이나? (활용 사례)

- **데이터 대시보드**: 매출/KPI 실시간 모니터링
- **ML 데모 앱**: 모델에 입력값을 넣고 예측 결과 확인
- **EDA 도구**: 데이터를 업로드하고 탐색적 분석 수행

- **사내 도구**: 비개발자도 사용할 수 있는 데이터 조회/필터링 앱
- **포트폴리오**: 프로젝트 결과물을 웹으로 공개

둘러보기: Streamlit Playground

> https://streamlit.io/playground

Streamlit 공식 사이트에서 **설치 없이 브라우저에서 바로** 코드를 수정하고 결과를 확인할 수 있는 Playground를 제공한다. 학생들과 함께 접속하여 샘플들을 둘러본다.

샘플	설명	주목 포인트
Blank	빈 템플릿	코드를 수정하면 우측 앱이 즉시 반영되는 것을 체험
Hello	인사말 + 풍선 애니메이션	`st.balloons()` 같은 재미있는 기능 확인
Charts	다양한 차트 + 20개 이상 위젯	오늘 배울 위젯들이 어떻게 동작하는지 미리보기
Dataframes	대규모 데이터 테이블, 스파클라인	인터랙티브 테이블의 정렬/검색 기능 체험
LLM Chat	채팅 앱	Streamlit으로 AI 챗봇도 만들 수 있음을 보여줌
Computer Vision	OpenCV 엣지 감지	이미지 처리 앱도 가능하다는 것을 확인
Geospatial	지도 시각화 (PyDeck, Folium)	지리 데이터 시각화의 가능성 확인

활동 방법:

1. https://streamlit.io/playground 접속
2. 좌측 코드 편집 → 우측 앱 자동 반영되는 구조를 확인
3. **Charts** 샘플에서 코드 일부를 직접 수정해보기 (예: 숫자 바꾸기, 텍스트 변경)
4. "playground를 통해 기능 등을 미리 검토 할 수 있다. 설치하지 않아도 간단히 확인 가능" → Streamlit의 간편함

> **강조**: "오늘 수업이 끝나면 여러분도 이런 앱을 직접 만들 수 있습니다!"

설치 확인

```
# 이미 설치되어 있어야 합니다 (1주차에 설치 완료)
pip install streamlit

# 설치 확인
streamlit --version
```

---

1. 한 장의 비교: Flask vs Streamlit (10분)

도입 질문

- "여러분이 데이터 분석 결과를 팀원에게 보여주고 싶다면 어떻게 하시겠습니까?"
- 일반적인 방법: Jupyter Notebook 공유, 엑셀 캡처, PPT 삽입...
- **문제점**: 상호작용이 불가능하고, 업데이트할 때마다 다시 공유해야 함

Flask로 웹 앱 만들기 (비교 대상)

> 🗀 소스 파일: `source/flask_app.py`, `source/templates/index.html`

```
# flask_app.py
from flask import Flask, render_template
import pandas as pd

app = Flask(__name__)

@app.route('/')
def index():
    df = pd.DataFrame({'이름': ['김철수', '이영희'], '점수': [85, 92]})
    return render_template('index.html', tables=[df.to_html()])

if __name__ == '__main__':
    app.run(debug=True)
```

```
<!-- templates/index.html -->
<!DOCTYPE html>
<html>
<body>
    <h1>학생 성적표</h1>
    {% for table in tables %}
        {{ table | safe }}
    {% endfor %}
</body>
</html>
```

- 필요한 것: Python 파일 + HTML 템플릿 파일 + 라우팅 설정
- 차트를 추가하려면? 보통 JavaScript 차트 라이브러리를 직접 붙이거나, Python 차트 결과를 이미지/HTML로 감싸 넣어야 해서 손이 많이 간다

Streamlit으로 같은 결과 만들기

> 🗀 소스 파일: `source/app.py`

```
# app.py
import streamlit as st
import pandas as pd

st.title('학생 성적표')
```

```
df = pd.DataFrame({'이름': ['김철수', '이영희'], '점수': [85, 92]})
st.dataframe(df)
st.bar_chart(df.set_index('이름'))
```

- **단 5줄**로 인터랙티브 테이블 + 차트까지 완성
- HTML, CSS, JavaScript 지식 불필요

핵심 메시지 전달

비교 항목	Flask	Streamlit
코드량	10줄 이상 + HTML	2~5줄
웹 지식 필요	HTML/CSS/JS 필수	불필요
차트 추가	JS 라이브러리 또는 Python 차트를 HTML로 감싸야 함	`st.line_chart()` 한 줄
실시간 반영	코드 변경 시 서버 재시작 또는 디버그 모드 설정 필요	별도 설정 없이 코드 저장만 해도 앱이 바로 갱신
대상 사용자	웹 개발자	**데이터 분석가**

> **"웹 개발자가 아니어도 데이터 앱을 배포할 수 있다"** — 이것이 Streamlit의 핵심 가치

실습: 첫 번째 Streamlit 앱 실행 (학생 직접 수행)

1. VSCode에서 `app.py` 파일 생성
2. 위 Streamlit 코드를 입력
3. 터미널에서 `streamlit run app.py` 실행
4. 브라우저에서 `http://localhost:8501` 확인

```
# 터미널에서 실행
streamlit run app.py
```

확인 포인트:

- 브라우저가 자동으로 열리는지 확인
- 테이블이 인터랙티브한지 확인 (컬럼 클릭으로 정렬해보기)
- 바 차트가 표시되는지 확인

> **응용 과제** (기본 실습을 완료한 학생용)
>
> 1. 학생 데이터를 5명 이상으로 늘리고, 영어, 수학 컬럼을 추가해보세요
> 2. `st.line_chart()`로 학생별 점수 추이 차트도 함께 표시해보세요
> 3. `st.table()`로 같은 데이터를 출력하고, `st.dataframe()`과 어떻게 다른지 비교해보세요

Tip: GitHub URL로 바로 실행하기

로컬에 파일이 없어도, GitHub에 올라간 Streamlit 앱을 **URL만으로 바로 실행**할 수 있다.

```
# GitHub Raw URL을 직접 지정하여 실행
streamlit run https://raw.githubusercontent.com/streamlit/demo-uber-nyc-pickups/master/streamlit_app.py
```

- 별도의 `git clone`이나 파일 다운로드 없이 즉시 실행 가능
- 다른 사람이 만든 Streamlit 앱을 빠르게 체험할 때 유용
- 위 예시는 **뉴욕 Uber 픽업 데이터 시각화** 데모 앱

---

2. 핵심 원리: Data Flow (20분)

개념 설명

Streamlit의 가장 독특하고 중요한 특징:

> **"사용자가 위젯을 조작하면, 스크립트가 상단부터 끝까지 전체 재실행된다"**

```
[사용자가 슬라이더를 움직임]
        ↓
[Python 스크립트 1번째 줄부터 재실행]
        ↓
[변경된 위젯 값이 반영된 결과 출력]
```

- 일반 웹 앱: 이벤트 핸들러가 특정 부분만 업데이트
- Streamlit: **전체 스크립트를 다시 실행**하여 결과를 갱신

실습 1: 재실행 원리 체험 — 버튼 클릭과 난수 (5분)

> 🗀 소스 파일: `source/data_flow_demo.py`

```python
# data_flow_demo.py
import streamlit as st
import numpy as np

st.title('Data Flow 체험')

# 버튼을 누를 때마다 스크립트가 재실행되므로 새로운 난수가 생성됨
st.write('랜덤 숫자:', np.random.randn(1))

st.button('다시 실행!')
```

체험 포인트:

- 버튼을 누를 때마다 랜덤 숫자가 바뀌는 것을 확인
- "왜 숫자가 바뀔까?" → 버튼 클릭 시 스크립트 **전체가 재실행**되기 때문
- 버튼 자체에 특별한 기능이 없어도, 클릭 = 재실행 트리거

실습 2: 재실행을 눈으로 확인하기 — 시각 + 위젯 (5분)

> 🗀 소스 파일: `source/data_flow_demo2.py`

```python
# data_flow_demo2.py
import streamlit as st
import numpy as np
from datetime import datetime

st.title('재실행 확인')

# 실행 시각을 출력하여 재실행을 눈으로 확인
st.write('현재 시각:', datetime.now().strftime('%H:%M:%S'))

name = st.text_input('이름을 입력하세요')
st.write(f'안녕하세요, {name}님!')

number = st.slider('숫자를 선택하세요', 0, 100, 50)
st.write(f'선택한 숫자: {number}')
```

체험 포인트:

- 이름을 입력하거나 슬라이더를 움직일 때마다 **시각이 갱신**됨
- 이것은 스크립트가 처음부터 다시 실행되었다는 증거

예상 질문: "스크립트가 처음부터 재실행되는데, 왜 입력한 이름과 슬라이더 값은 초기화되지 않나요?"

> Streamlit의 **위젯**(`st.text_input`, `st.slider` 등)은 내부적으로 자체 상태 관리를 한다. 각 위젯에 **고유 키(key)**를 부여하고, 현재 값을 `st.session_state`에 자동 저장한다.
>
> ```
> [슬라이더를 70으로 변경]
>  ↓
> [Streamlit이 내부적으로 슬라이더 값 70을 session_state에 저장]
>  ↓
> [스크립트 전체 재실행]
>  ↓
> [st.slider()가 실행될 때 session_state에서 70을 꺼내서 기본값으로 사용]
>  ↓
> [number = 70 반환]
> ```

구분	재실행 시 동작	위 코드에서의 예시
일반 변수	초기화됨	`datetime.now()` → 매번 새로운 시각

구분	재실행 시 동작	위 코드에서의 예시
위젯 반환값	Streamlit이 기억함	`st.text_input()` → 입력한 이름 유지, `st.slider()` → 선택한 숫자 유지

> 이 차이가 바로 다음 실습(카운터)에서 문제가 되는 핵심 원인이다.

실습 3: 재실행의 함정 — 카운터가 안 되는 이유 (5분)

> 클릭하면 숫자가 1씩 증가하는 카운터를 만들어보세요

> 🗀 소스 파일: `source/counter_broken.py` (실패), `source/counter_fixed.py` (성공)

```
# counter_broken.py
import streamlit as st

st.title('카운터 (실패 버전)')

count = 0

if st.button('클릭! (+1)'):
    count += 1

st.write(f'현재 카운트: {count}')
```

1. 버튼을 여러 번 클릭해보기
2. "왜 카운트가 계속 0 또는 1일까?"

설명:

- 버튼을 클릭하면 스크립트가 **처음부터** 재실행
- `count = 0`이 매번 다시 실행되므로 값이 초기화됨
- 이것이 Data Flow의 **한계** — 일반 변수는 재실행 시 유지되지 않음

해결책 맛보기 (session_state):

```
# counter_fixed.py
import streamlit as st

st.title('카운터 (성공 버전)')

# session_state: 재실행되어도 값이 유지되는 특별한 저장소
if 'count' not in st.session_state:
    st.session_state.count = 0

if st.button('클릭! (+1)'):
    st.session_state.count += 1

st.write(f'현재 카운트: {st.session_state.count}')
```

> "session_state는 다음 목록에서 자세히 다룹니다. 지금은 '재실행되어도 값을 기억하는 방법이 있다'는 것만 알아두세요."

> **응용 과제** (기본 실습을 완료한 학생용)
>
> 1. 카운터에 "감소" 버튼을 추가하여 `+1` / `-1` 두 개 버튼이 동작하도록 만들어보세요
> 2. "리셋" 버튼을 추가하여 카운트를 0으로 초기화하는 기능을 구현해보세요
> 3. (힌트: `st.columns(3)`으로 버튼 3개를 가로로 배치하면 더 깔끔합니다)

Data Flow 정리

장점	단점
코드가 단순해짐 (위→아래 순서)	매번 전체 재실행 → 무거운 연산 반복
상태 관리 걱정 없음	대용량 데이터 로딩이 반복될 수 있음
디버깅이 쉬움	일반 변수는 재실행 시 초기화됨

문제	해결책	다루는 시기
무거운 연산 반복	`@st.cache_data`	4주차
변수 값 초기화	`st.session_state`	4주차

---

3. st.write()와 Magic Commands (15분)

st.write() — Streamlit의 만능 출력 함수 (7분)

`st.write()`는 넘겨주는 데이터 타입에 따라 **자동으로** 적절한 형태로 출력한다.

실습: 다양한 데이터 타입 출력

> 🗀 소스 파일: `source/write_demo.py`

```python
# write_demo.py
import streamlit as st
import pandas as pd
import numpy as np

st.title('st.write() 만능 출력기')

# 1. 문자열 → 텍스트로 출력
st.write('안녕하세요, Streamlit!')

# 2. 숫자 → 그대로 출력
st.write(42)
st.write(3.14)

# 3. 마크다운 → 렌더링
st.write('## 이것은 제목입니다')
st.write('**굵은 글씨**, *기울임*, `코드`')
```

```
# 4. DataFrame → 인터랙티브 테이블
df = pd.DataFrame({
    '이름': ['김철수', '이영희', '박민수'],
    '국어': [90, 85, 78],
    '영어': [88, 92, 95],
    '수학': [76, 88, 90]
})
st.write(df)

# 5. 딕셔너리 → JSON 형태
st.write({'이름': '김철수', '나이': 22, '전공': '컴퓨터공학'})

# 6. 리스트 → 배열 형태
st.write([1, 2, 3, 4, 5])

# 7. 여러 인자를 한번에
st.write('김철수의 점수는', 90, '점입니다')
```

핵심 포인트:

- `st.write()`는 타입을 보고 자동으로 최적의 형태로 출력
- `print()`의 Streamlit 버전이라고 생각하면 됨
- 문자열에 마크다운 문법을 쓰면 자동으로 렌더링

st.write() vs 전용 함수 비교

용도	st.write()	전용 함수	차이점
텍스트	`st.write('텍스트')`	`st.text('텍스트')`	text는 마크다운 렌더링 안 함
마크다운	`st.write('**굵게**')`	`st.markdown('**굵게**')`	동일한 결과
DataFrame	`st.write(df)`	`st.dataframe(df)`	dataframe은 높이/너비 조절 가능
차트	—	`st.line_chart(df)`	write로는 차트 불가

> "대부분 `st.write()`로 충분하지만, 세밀한 제어가 필요하면 전용 함수를 쓰세요."

텍스트 출력 함수 한눈에 보기

함수	용도	예시	결과
`st.title()`	페이지 제목 (가장 큰 글씨)	`st.title('대시보드')`	**대시보드** (h1)
`st.header()`	섹션 제목	`st.header('1. 개요')`	**1. 개요** (h2)
`st.subheader()`	소제목	`st.subheader('요약')`	**요약** (h3)

함수	용도	예시	결과
`st.text()`	고정폭 텍스트 (마크다운 X)	`st.text('plain text')`	`plain text`
`st.markdown()`	마크다운 렌더링	`st.markdown('**굵게**')`	**굵게**
`st.caption()`	작은 회색 설명 텍스트	`st.caption('출처: ...')`	출처: ...
`st.code()`	코드 블록 (구문 강조)	`st.code('print("hi")', language='python')`	코드 블록
`st.latex()`	수학 수식 (LaTeX)	`st.latex(r'\sigma(x) = \frac{1}{1 + e^{-x}}')`	Sigmoid 함수
`st.write()`	**만능 출력** (타입 자동 감지)	`st.write(df)`	타입에 맞게 출력

> `st.write()`는 내부적으로 위 함수들을 자동 선택합니다. 명시적으로 특정 스타일이 필요할 때 전용 함수를 사용하세요.

Magic Commands — st.write() 없이 출력하기 (3분)

Streamlit에서는 `st.write()`를 호출하지 않아도 **변수명이나 문자열을 단독으로 작성**하면 자동으로 화면에 출력된다.

> 🗀 소스 파일: `source/magic_demo.py`

```
# magic_demo.py
import streamlit as st
import pandas as pd

st.title('Magic Commands 체험')

# 방법 1: st.write() 사용
df = pd.DataFrame({
    '과목': ['국어', '영어', '수학'],
    '점수': [90, 85, 95]
})
st.write(df)  # 명시적 호출

# 방법 2: Magic! - 변수명만 적으면 자동 출력
df2 = pd.DataFrame({
    '과목': ['과학', '사회', '역사'],
    '점수': [88, 76, 92]
})
df2  # st.write(df2)와 동일한 결과!

# 방법 3: 삼중 따옴표 마크다운도 Magic으로 렌더링
"""
## 오늘의 분석 결과
- 매출이 전월 대비 **15% 증가**
```

```
- 신규 고객 수 *200명* 달성
> 전반적으로 긍정적인 추세입니다.
"""
```

체험 포인트:

- `st.write(df)`와 `df`만 적은 것의 결과가 동일한지 확인
- 삼중 따옴표(`"""..."""`) 안에 마크다운을 적으면 그대로 렌더링
- Magic은 편리하지만, **명시적인 `st.write()`를 쓰는 것이 가독성이 더 좋음**
- **실무 조언**: 빠른 확인이나 디버깅에는 Magic이 유용하지만, 팀 협업이나 유지보수가 필요한 프로젝트에서는 `st.write()`나 `st.dataframe()`처럼 **명시적인 함수를 사용하는 것을 권장**한다. 코드를 읽는 사람이 "이 줄이 출력을 의도한 것인지" 바로 알 수 있기 때문이다.

종합 실습: 자기소개 페이지 만들기 (5분)

> 지금까지 배운 것을 조합하여 자기소개 웹 페이지를 만들어 봅시다.

> 🗀 소스 파일: `source/my_profile.py`

```python
# my_profile.py
import streamlit as st
import pandas as pd

st.title('자기소개')

# 본인 정보 수정하기!
st.write('## 기본 정보')
st.write('**이름**: 홍길동')
st.write('**학과**: 인공지능소프트웨어학과')
st.write('**학년**: 3학년')

st.write('---')  # 구분선

st.write('## 이번 학기 시간표')
schedule = pd.DataFrame({
    '요일': ['월', '화', '수', '목', '금'],
    '2교시': ['데이터베이스', '-', '알고리즘', '-', '빅데이터분석프로젝트'],
    '3교시': ['데이터베이스', '웹프로그래밍', '알고리즘', '웹프로그래밍', '빅데이터분석
프로젝트'],
    '4교시': ['-', '웹프로그래밍', '-', '웹프로그래밍', '빅데이터분석프로젝트']
})
st.write(schedule)

st.write('---')

st.write('## 관심 분야')
st.write('- 머신러닝')
st.write('- 데이터 시각화')
st.write('- 웹 개발')

st.write('---')
```

```
st.write('## 이번 학기 목표')
goals = pd.DataFrame({
    '목표': ['Streamlit 마스터', 'HuggingFace 활용', '포트폴리오 완성'],
    '달성률': [10, 0, 5]
})
st.write(goals)
st.bar_chart(goals.set_index('목표'))
```

학생 활동:

1. 위 코드를 `my_profile.py`로 저장
2. **본인의 정보로 내용을 수정**
3. `streamlit run my_profile.py`로 실행
4. 코드를 수정하고 저장한 후 브라우저에서 **자동 반영**되는지 확인

확인해야 할 것:

- 코드를 저장하면 브라우저 우측 상단에 "Rerun" 버튼이 나타남
- "Always rerun" 을 체크하면 저장할 때마다 자동 반영
- 이것이 Streamlit의 **Data Flow** — 코드 변경 → 전체 재실행 → 화면 갱신

> **응용 과제** (기본 실습을 완료한 학생용)
>
> 1. `st.header()`, `st.subheader()`, `st.caption()` 등을 활용하여 페이지 구조를 더 풍부하게 만들어보세요
> 2. 달성률을 `st.bar_chart()` 대신 `st.metric()`으로 표시하고, `delta`에 "목표 대비" 값을 넣어보세요
> 3. Magic Commands(`"""..."""`)로 마크다운 자기소개 섹션을 추가해보세요

1교시 마무리 체크리스트

- ☐ Streamlit이 Flask 대비 어떤 장점이 있는지 설명할 수 있다
- ☐ `streamlit run 파일명.py` 명령어로 앱을 실행할 수 있다
- ☐ Data Flow(재실행) 원리를 이해하고, 왜 숫자가 바뀌는지 설명할 수 있다
- ☐ 카운터가 안 되는 이유를 Data Flow로 설명할 수 있다
- ☐ st.write()로 문자열, DataFrame, 딕셔너리 등 다양한 타입을 출력할 수 있다
- ☐ Magic Commands로 변수를 출력하고 마크다운을 렌더링할 수 있다